REPORT

Title

Alpha-Beta Pruning Algorithm for Gaming

Aim

To implement the **Alpha-Beta Pruning algorithm** for a two-player gaming application using Python and reduce unnecessary game-tree searches.

Problem Statement

In two-player games, the number of possible moves increases rapidly as the game progresses. The Minimax algorithm explores these possible moves to determine the best move, but it may evaluate many branches that do not influence the final decision.

Alpha-Beta Pruning improves Minimax by eliminating such unnecessary branches. The objective is to implement Alpha-Beta Pruning in a Tic-Tac-Toe game and enable the computer to select an optimal move.

Requirements

- Python 3.x
- Python IDLE / VS Code / any Python IDE
- Basic knowledge of Python
- Knowledge of Minimax algorithm
- Understanding of game trees

Procedure

1. Create a 3 × 3 Tic-Tac-Toe board.

2. Assign X to the human player and O to the computer.
3. Display the current board.
4. Accept a valid move from the human player.
5. Check whether the human player has won.
6. Generate all possible moves for the computer.
7. Apply the Minimax algorithm to evaluate the possible moves.
8. Initialize Alpha to negative infinity and Beta to positive infinity.
9. Update Alpha for the maximizing player.
10. Update Beta for the minimizing player.
11. If Alpha becomes greater than or equal to Beta, prune the remaining branches.
12. Select the move with the best score for the computer.
13. Continue the game until a player wins or the game ends in a draw.

Program

```
HUMAN = "X"
AI = "O"
EMPTY = " "

board = [EMPTY] * 9

def check_winner(player):
    winning_positions = [
        (0,1,2), (3,4,5), (6,7,8),
        (0,3,6), (1,4,7), (2,5,8),
        (0,4,8), (2,4,6)
    ]

    return any(
        board[a] == board[b] == board[c] == player
        for a, b, c in winning_positions
    )

def is_draw():
    return EMPTY not in board

def minimax(alpha, beta, maximizing):

    if check_winner(AI):
        return 1

    if check_winner(HUMAN):
        return -1

    if is_draw():
```

```
            return 0

        if maximizing:
            best_score = -float("inf")

            for i in range(9):
                if board[i] == EMPTY:
                    board[i] = AI
                    score = minimax(alpha, beta, False)
                    board[i] = EMPTY

                    best_score = max(best_score, score)
                    alpha = max(alpha, best_score)

                    if alpha >= beta:
                        break

            return best_score

        else:
            best_score = float("inf")

            for i in range(9):
                if board[i] == EMPTY:
                    board[i] = HUMAN
                    score = minimax(alpha, beta, True)
                    board[i] = EMPTY

                    best_score = min(best_score, score)
                    beta = min(beta, best_score)

                    if alpha >= beta:
                        break

            return best_score

    def best_move():
        best_score = -float("inf")
        move = -1

        for i in range(9):
            if board[i] == EMPTY:
                board[i] = AI

                score = minimax(
                    -float("inf"),
                    float("inf"),
                    False
                )

                board[i] = EMPTY

                if score > best_score:
                    best_score = score
                    move = i

        return move
```

Sample Output

```
TIC-TAC-TOE
You = X
Computer = O

X | X |
--+---+--
  | O |
--+---+--
  |   |

Computer chooses position: 3

Computer wins!
```

Conclusion

The Alpha-Beta Pruning algorithm was successfully implemented for a Tic-Tac-Toe gaming application. It improves the Minimax algorithm by pruning branches that cannot affect the final decision. This reduces unnecessary computations while maintaining the optimal decision-making capability of Minimax.